



ScholarNest

Spark Azure Databricks

Databricks Spark Certification and beyond

Instructor: Prashant Kumar Pandey



Azure
Databricks



Absolute Beginner to Specialization in Apache Spark and Azure Databricks



Creating Data Frames Programmatically

We learned that Spark session is the starting point. However, Spark programming is all about reading, processing, and writing data. And Spark Dataframe is your tool for doing all of this.

But how do you create a Spark Dataframe?

We have many ways to create a Spark Dataframe. Here is a list of some approaches:

1. read
2. sql()
3. table()
4. range()
5. createDataFrame()

All of these are offered by SparkSession.

We already learned about the Spark Session read attribute. Here is the code shown below that we used in an earlier example. The *SparkSession.read* gives you a *DataframeReader*, and you can use it to read data from a variety of data sources. We learned to read data from a CSV file, but *DataframeReader* supports many other sources.

Creating Spark Data Frame

- **read**
- `sql()`
- `table()`
- `range()`
- `createDataFrame()`

```
raw_fire_df = spark.read \
    .format("csv") \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .load("/databricks-datasets/learning-spark-v2/sf-fire/sf-fire-calls.csv")
```

We also learned to use the *SparkSession.sql()* method. Here is the earlier example. So you can use a Spark SQL in the *SparkSession.sql()* method, and the result of the SQL is a Spark Dataframe.

Creating Spark Data Frame

- read
- **sql()**
- table()
- range()
- createDataFrame()

```
q1_sql_df = spark.sql("""
    select count(distinct CallType) as distinct_call_type_count
    from fire_service_calls_view
    where CallType is not null
    """)
```

The third method is even more straightforward than the SQL() method. You can use *spark.table()* method and pass in a table name to create a Dataframe. And we have an example code shown below. So you can give a database and table name to read a Spark table and create a Dataframe out of it. So if someone is asking you to convert a Spark table to a Spark Dataframe, you can use *spark.table()* method.

Creating Spark Data Frame

- read
- sql()
- **table()**
- range()
- createDataFrame()

```
df1 = spark.table("spark_db.table_name")
```

The main focus of this document was to help you understand the `range()` and the `createDataFrame()` methods.

These two methods allow you to create a Dataframe without reading data from a source.

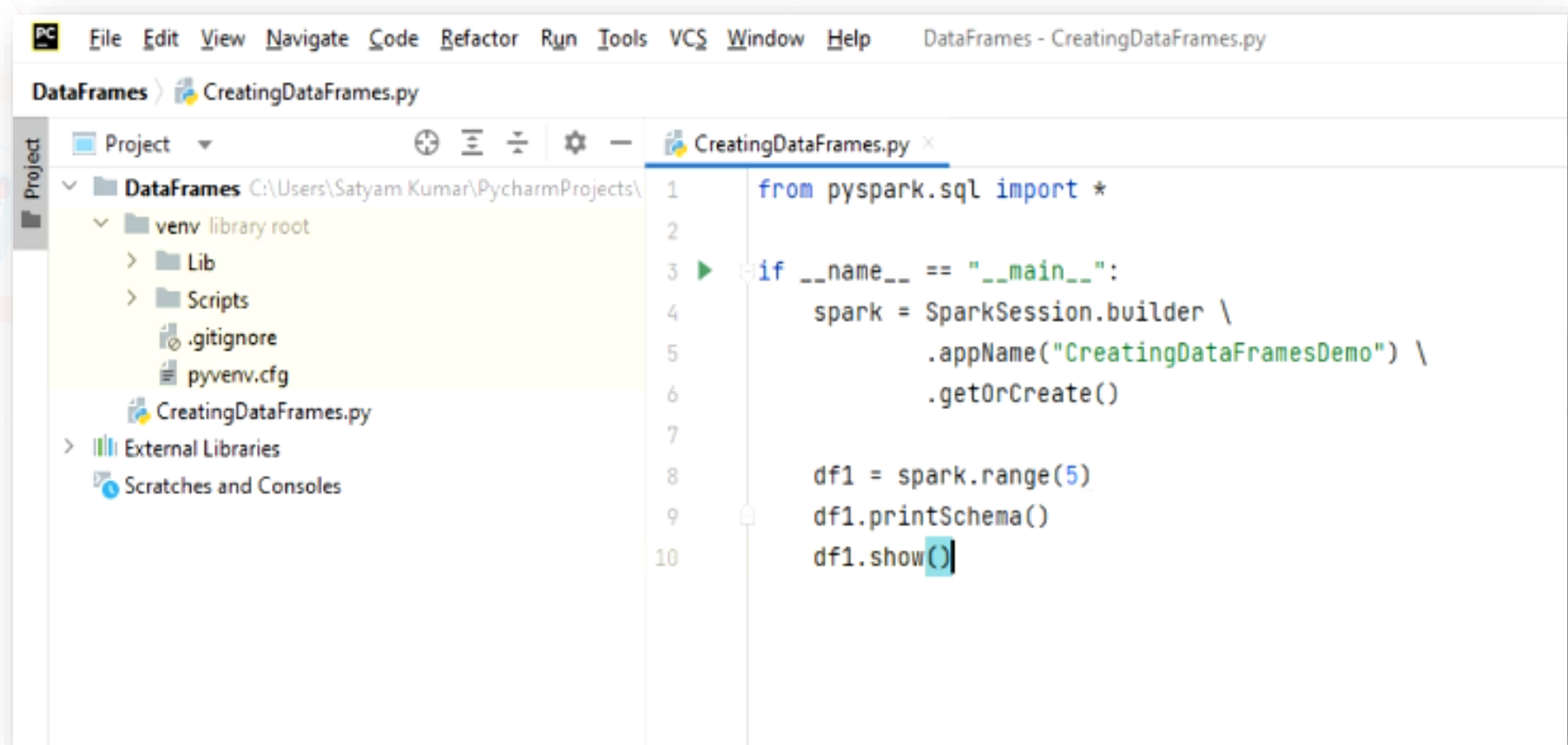
Creating Spark Data Frame

- `read`
- `sql()`
- `table()`
- `range()` ←
- `createDataFrame()` ←

Here I have opened a previous project which I created earlier. **(Reference – DataFrames)**

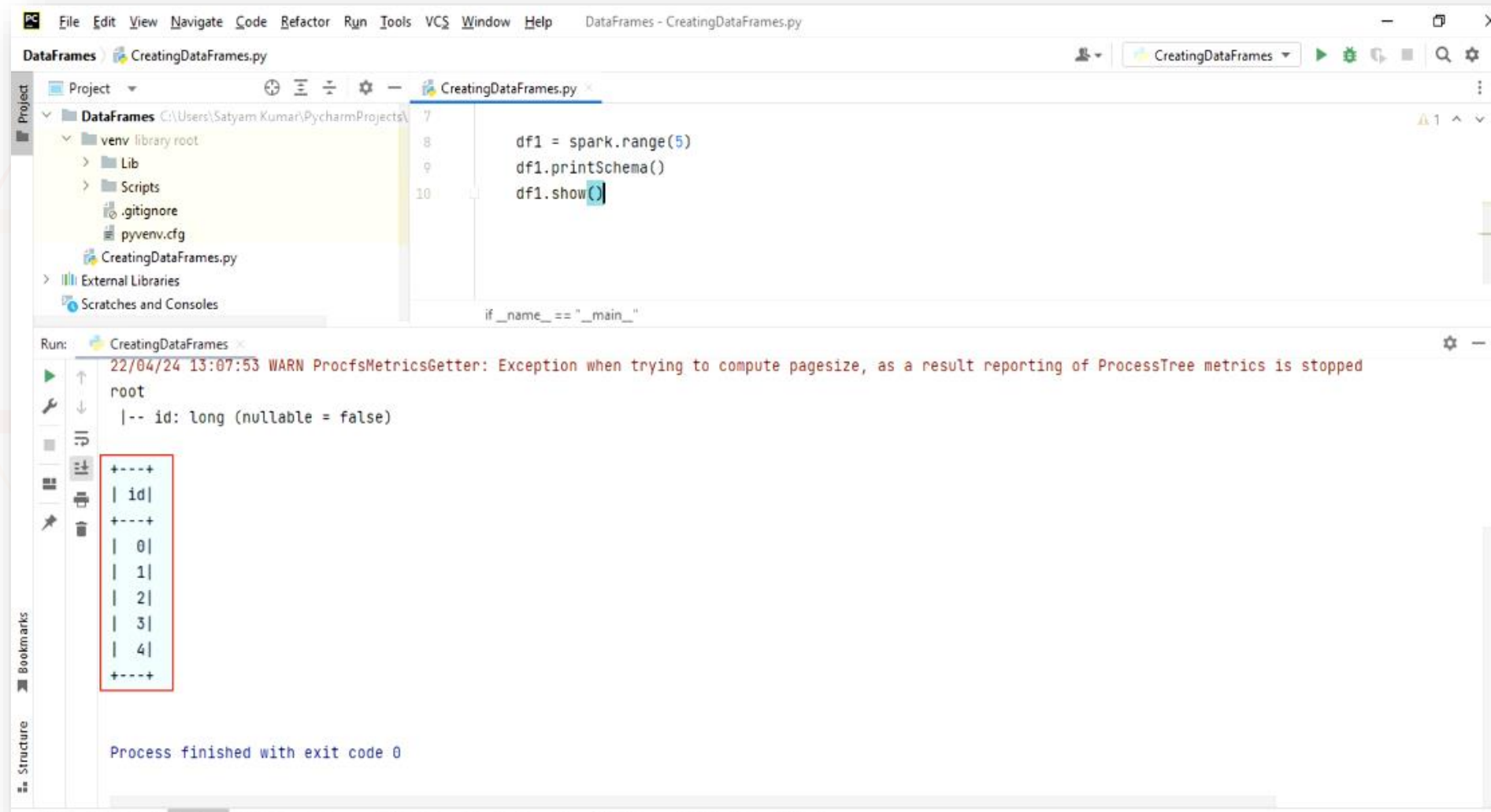
In this example, we learned to create a Spark Session. Now, the next step is to create a Data Frame.

I have written the code for that. We have a single-column Dataframe. The *spark.range()* allows you to create a single-column Dataframe. You might need a single-column data frame for testing something. And *spark.range()* is an easy method to create it. The range() method takes many parameters like start, end, step, and the number of partitions. You can show this Dataframe and check the schema.

A screenshot of an IDE window titled 'DataFrames - CreatingDataFrames.py'. The left sidebar shows a project tree with folders 'DataFrames' and 'venv library root', and files '.gitignore', 'pyvenv.cfg', and 'CreatingDataFrames.py'. The main editor area shows the following Python code:

```
1 from pyspark.sql import *
2
3 if __name__ == "__main__":
4     spark = SparkSession.builder \
5         .appName("CreatingDataFramesDemo") \
6         .getOrCreate()
7
8     df1 = spark.range(5)
9     df1.printSchema()
10    df1.show()
```

If we run the code, we can see the results as shown below.
By default, the column name is id, and its data type is long. The Dataframe values start from zero and stop at 5.



The screenshot shows the PyCharm IDE with a project named 'DataFrames'. The file 'CreatingDataFrames.py' is open, containing the following code:

```
df1 = spark.range(5)
df1.printSchema()
df1.show()
```

The output of the code execution is displayed in the 'Run' console. It shows a warning message and the schema of the DataFrame:

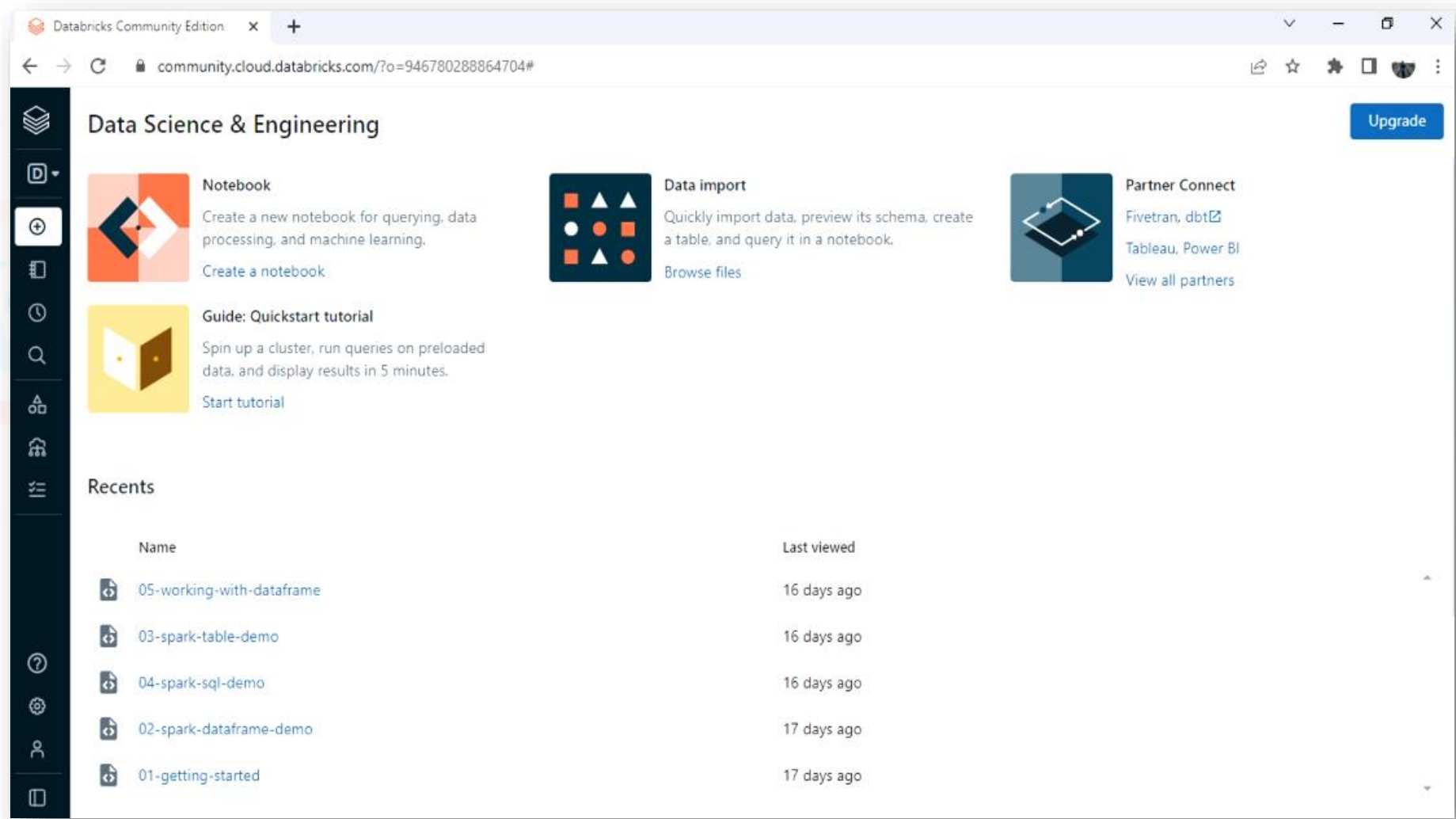
```
22/04/24 13:07:53 WARN ProcsMetricsGetter: Exception when trying to compute pagesize, as a result reporting of ProcessTree metrics is stopped
root
 |-- id: long (nullable = false)
```

The data rows are displayed in a table format, with the 'id' column highlighted by a red box:

id
0
1
2
3
4

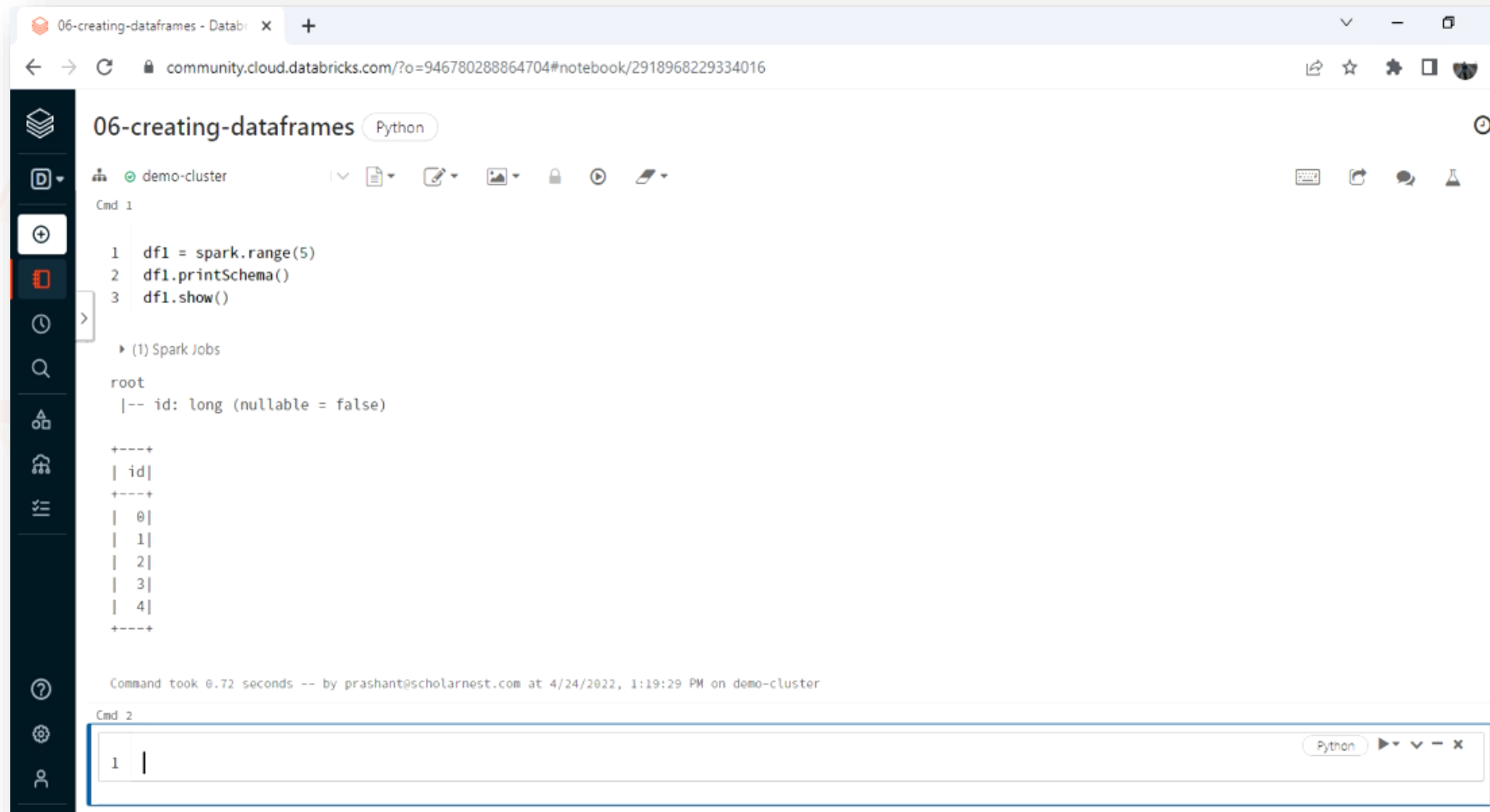
The process finished with exit code 0.

If you find it difficult to work with IDE, you can switch to Notebooks.
Here I am in my Databricks Workspace.
Create a new notebook. **(Reference - 06-creating-dataframes.ipynb)**



Now you can go back to the IDE and copy the last three lines of the code.

Databricks notebook automatically creates a Spark session variable for me. So I do not need to create a Spark session variable. If you run the cell, you will get the same output.



The screenshot shows a Databricks notebook titled "06-creating-dataframes" in the Python language. The notebook is running on a "demo-cluster". The code cell contains three lines of Python code:

```
1 df1 = spark.range(5)
2 df1.printSchema()
3 df1.show()
```

The output of the code cell is displayed below the code. It shows the Spark job information, the root schema, and the resulting data:

```
(1) Spark Jobs
root
 |-- id: long (nullable = false)

+----+
| id |
+----+
|  0 |
|  1 |
|  2 |
|  3 |
|  4 |
+----+
```

Below the output, a status message indicates: "Command took 0.72 seconds -- by prashant@scholarnest.com at 4/24/2022, 1:19:29 PM on demo-cluster". At the bottom of the notebook, there is a new code cell labeled "Cmd 2" with a single line of code:

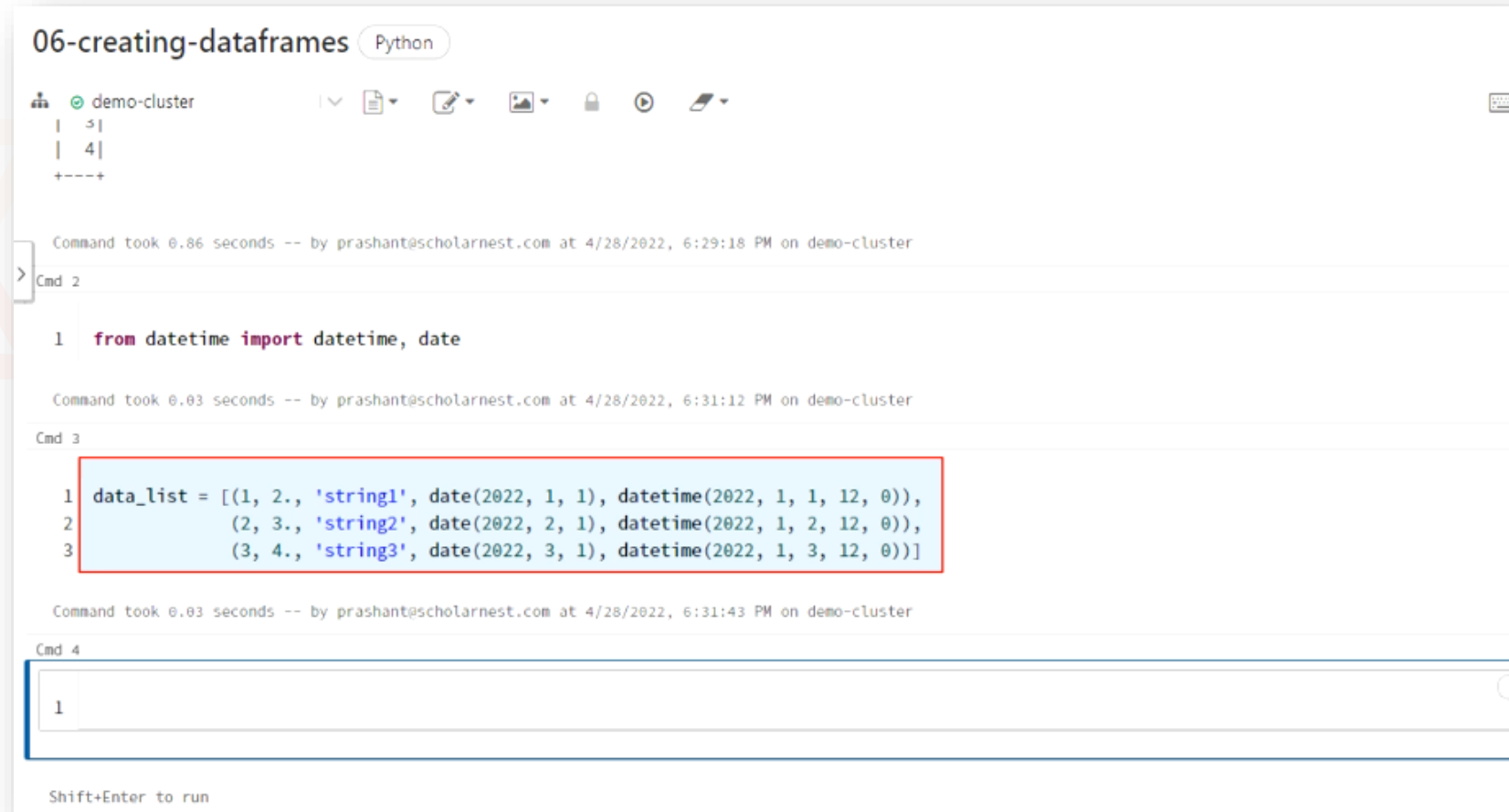
```
1 |
```

The range method is basic. It only supports creating a single-column Dataframe. But we also have *spark.createDataFrame()* method. It converts a list into a Spark Dataframe. Look at the requirement given in the image below to create a Dataframe programmatically. You have column names shown here. Then you have some rows shown in the diagram. You also expect column data types, as shown at the top.

long	double	string	date	timestamp	
a	b	c	d	e	
1	2.0	string1	2000-01-01	2000-01-01	12:00:00
2	3.0	string2	2000-02-01	2000-01-02	12:00:00
3	4.0	string3	2000-03-01	2000-01-03	12:00:00

The `createDataFrame()` takes a Python list and converts it into a Dataframe. We define a python list inside a square brackets. Then we define each record as a tuple inside a pair of parenthesis. I wanted three records, so I defined three types inside the square bracket.

The column is a date, and the last one is a datetime, so I am using Python date and datetime functions from the Python datetime module. You must import the date and datetime functions from the datetime module.



```
06-creating-dataframes Python

demo-cluster
| 3|
| 4|
+---+

Command took 0.86 seconds -- by prashant@scholarnest.com at 4/28/2022, 6:29:18 PM on demo-cluster

> Cmd 2

1 from datetime import datetime, date

Command took 0.03 seconds -- by prashant@scholarnest.com at 4/28/2022, 6:31:12 PM on demo-cluster

Cmd 3

1 data_list = [(1, 2., 'string1', date(2022, 1, 1), datetime(2022, 1, 1, 12, 0)),
2              (2, 3., 'string2', date(2022, 2, 1), datetime(2022, 1, 2, 12, 0)),
3              (3, 4., 'string3', date(2022, 3, 1), datetime(2022, 1, 3, 12, 0))]

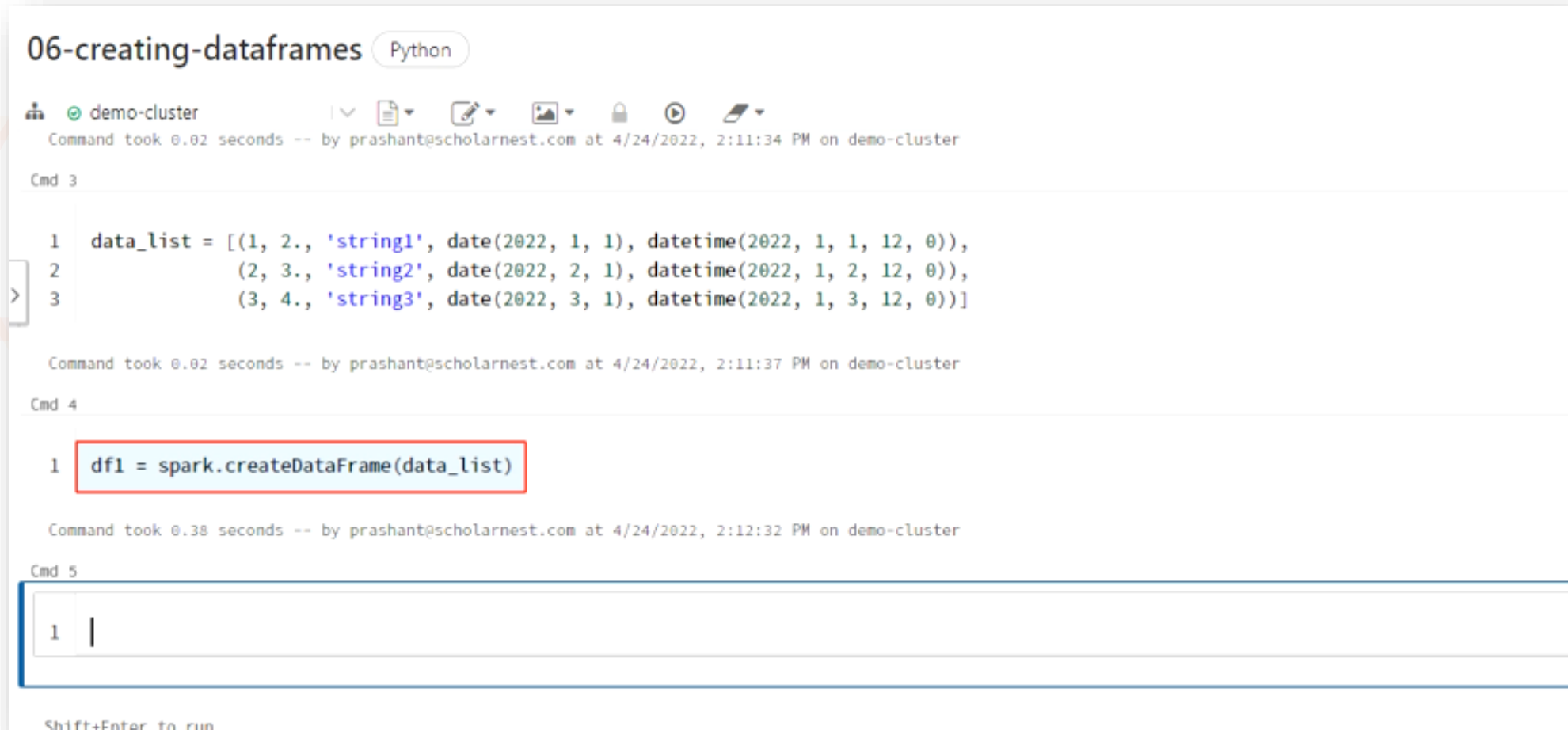
Command took 0.03 seconds -- by prashant@scholarnest.com at 4/28/2022, 6:31:43 PM on demo-cluster

Cmd 4

1

Shift+Enter to run
```

Once you have the list, you can create a Dataframe as shown below. I am using `spark.createDataFrame()` method and passing the `data_list`. The `createDataFrame()` method will convert the list into a Spark Dataframe and return it. So the `df1` is my Spark Dataframe.



The screenshot shows a Jupyter Notebook titled "06-creating-dataframes" with a "Python" language selector. The interface includes a toolbar with icons for saving, running, and other actions. Below the toolbar, the command history shows three commands. Command 3 defines a list named `data_list` with three rows of data. Command 4, which is highlighted with a red box, uses `spark.createDataFrame(data_list)` to create a DataFrame. Command 5 is currently being edited in the code cell.

```
06-creating-dataframes Python

demo-cluster
Command took 0.02 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:11:34 PM on demo-cluster

Cmd 3

1 data_list = [(1, 2., 'string1', date(2022, 1, 1), datetime(2022, 1, 1, 12, 0)),
2             (2, 3., 'string2', date(2022, 2, 1), datetime(2022, 1, 2, 12, 0)),
3             (3, 4., 'string3', date(2022, 3, 1), datetime(2022, 1, 3, 12, 0))]

Command took 0.02 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:11:37 PM on demo-cluster

Cmd 4

1 df1 = spark.createDataFrame(data_list)

Command took 0.38 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:12:32 PM on demo-cluster

Cmd 5

1 |

Shift+Enter to run
```

You can show the Dataframe and check if you got the desired output. Adjacent to the output is an image of the desired Dataframe. I am almost there except for a tiny problem. The column names are not as per my requirement. It took some default column names here. But I wanted to give some different column names. I will fix this in a while, but before that let us check the schema.

06-creating-dataframes Python

demo-cluster

Cmd 4

1 df1 = spark.createDataFrame

Command took 0.38 seconds -- by prashant

Cmd 5

1 df1.show()

(3) Spark Jobs

1|2.0|string1|2022-01-01|2022-01-01 12:00:00|

2|3.0|string2|2022-02-01|2022-01-02 12:00:00|

3|4.0|string3|2022-03-01|2022-01-03 12:00:00|

Command took 1.28 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:12:55 PM on demo-cluster

long	double	string	date	timestamp
a	b	c	d	e
1	2.0	string1	2000-01-01	2000-01-01 12:00:00
2	3.0	string2	2000-02-01	2000-01-02 12:00:00
3	4.0	string3	2000-03-01	2000-01-03 12:00:00

Here is the schema shown below. It looks like data types are fine. The *createDataFrame* automatically infers the schema of the Dataframe based on the given data. So do remember to define your data list with correct data types so the *createDataFrame* can easily infer it.

06-creating-dataframes Python

demo-cluster

1	2.0	string1	2022-01-01	2022-01-01 12:00:00
2	3.0	string2	2022-02-01	2022-01-02 12:00:00
3	4.0	string3	2022-03-01	2022-01-03 12:00:00

Command took 1.28 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:12:55 PM on demo-cluster

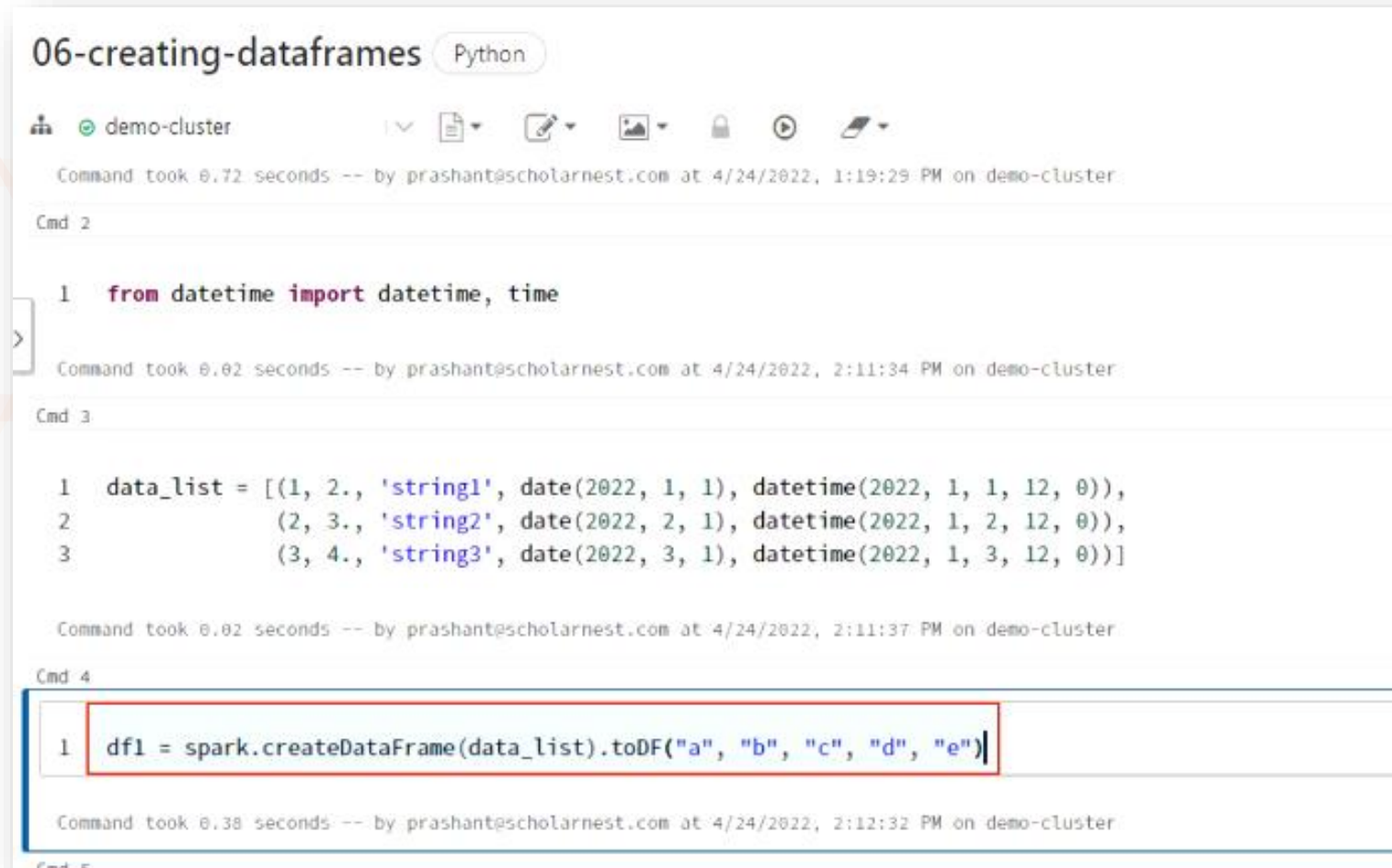
Cmd 6

```
1 df1.printSchema()
```

```
root
|-- _1: long (nullable = true)
|-- _2: double (nullable = true)
|-- _3: string (nullable = true)
|-- _4: date (nullable = true)
|-- _5: timestamp (nullable = true)
```

Command took 0.03 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:13:38 PM on demo-cluster

Now let's come back to the column name problem. How do I fix these column names? One easy way is to use the *toDF()* method. The *toDF()* is a Dataframe transformation. So once you have the Dataframe, you can use the *toDF()* to change all the column names at once. You can chain it after the *createDataFrame()* as shown below.



The screenshot shows a Jupyter Notebook titled "06-creating-dataframes" in a Python environment. It displays four command blocks. Command 2 imports datetime and time. Command 3 creates a list of data rows. Command 4, which is highlighted with a red box, uses spark.createDataFrame() to create a DataFrame from the list and then chains .toDF() with new column names "a", "b", "c", "d", and "e".

```
06-creating-dataframes Python

demo-cluster

Command took 0.72 seconds -- by prashant@scholarnest.com at 4/24/2022, 1:19:29 PM on demo-cluster

Cmd 2

1 from datetime import datetime, time

Command took 0.02 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:11:34 PM on demo-cluster

Cmd 3

1 data_list = [(1, 2., 'string1', date(2022, 1, 1), datetime(2022, 1, 1, 12, 0)),
2             (2, 3., 'string2', date(2022, 2, 1), datetime(2022, 1, 2, 12, 0)),
3             (3, 4., 'string3', date(2022, 3, 1), datetime(2022, 1, 3, 12, 0))]

Command took 0.02 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:11:37 PM on demo-cluster

Cmd 4

1 df1 = spark.createDataFrame(data_list).toDF("a", "b", "c", "d", "e")

Command took 0.38 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:12:32 PM on demo-cluster

Cmd 5
```

You can again check your Dataframe, and the column names are rectified as desired.

06-creating-dataframes Python

demo-cluster
Command took 0.09 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:15:17 PM on demo-cluster

Cmd 5

```
1 df1.show()
```

> (3) Spark Jobs

	a	b	c	d	e
1	2.0	string1	2022-01-01	2022-01-01 12:00:00	
2	3.0	string2	2022-02-01	2022-01-02 12:00:00	
3	4.0	string3	2022-03-01	2022-01-03 12:00:00	

Command took 0.52 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:15:21 PM on demo-cluster

Cmd 6

We created a *data_list* and used it to create a Dataframe. The *createDataFrame()* automatically infers the schema and generates some default column names. But you can supply a schema definition and enforce it. And here are two ways to define a schema as shown below.

The first one is simply a list of column names.

The second approach defines the column names and data types both.

06-creating-dataframes

Python



demo-cluster



```
-- c: string (nullable = true)
-- d: date (nullable = true)
-- e: timestamp (nullable = true)
```

Command took 0.04 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:15:39 PM on demo-cluster



Cmd 7

```
1 schema_1 = ['a', 'b', 'c', 'd', 'e']
2 schema_2 = 'a int, b double, c string, d date, e timestamp'
```

Command took 0.04 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:19:38 PM on demo-cluster

I am using the first schema to show you the result.

I supplied a list of column names, so the *createDataFrame* should use the given columns names instead of the default names. Data types are still correct because the *createDataFrame()* could infer it correctly from the *data_list*.

06-creating-dataframes

Python

demo-cluster

Command took 0.04 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:19:38 PM on demo-cluster

Cmd 7

```
1 schema_1 = ['a', 'b', 'c', 'd', 'e']
2 schema_2 = 'a int, b double, c string, d date, e timestamp'
```

Command took 0.04 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:19:38 PM on demo-cluster

Cmd 8

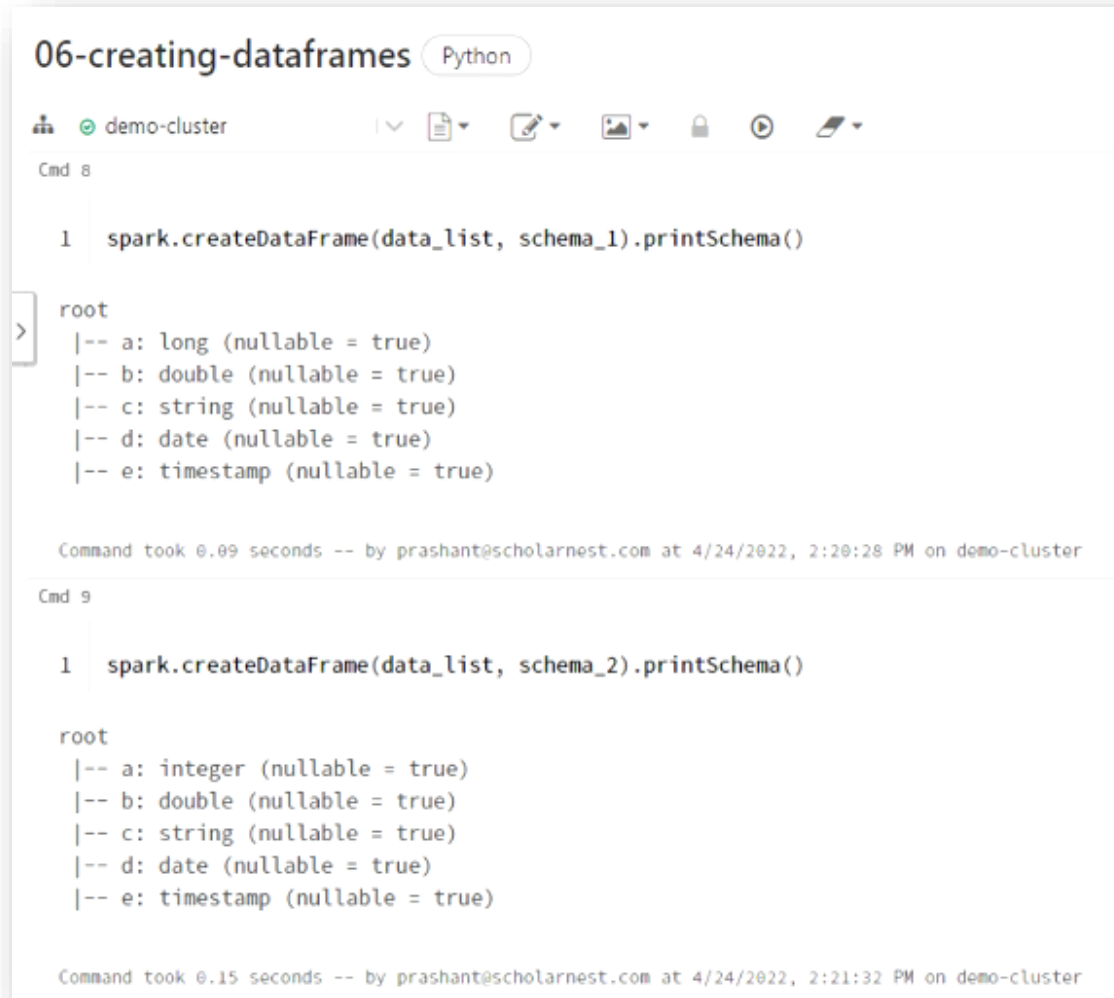
```
1 spark.createDataFrame(data_list, schema_1).printSchema()
```

root

```
-- a: long (nullable = true)
-- b: double (nullable = true)
-- c: string (nullable = true)
-- d: date (nullable = true)
-- e: timestamp (nullable = true)
```

Command took 0.09 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:20:28 PM on demo-cluster

Then in the second schema, you can see that I purposefully change the data type for the first column. I know the *createDataFrame* will infer the first column to be a long value. But I want it to be an integer. I tried enforcing a schema and stopped *createDataFrame* from inferring it, and it worked. You can see the output in the bottom section of the image below.



```
06-creating-dataframes Python

demo-cluster

Cmd 8

1 spark.createDataFrame(data_list, schema_1).printSchema()

root
|-- a: long (nullable = true)
|-- b: double (nullable = true)
|-- c: string (nullable = true)
|-- d: date (nullable = true)
|-- e: timestamp (nullable = true)

Command took 0.09 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:20:28 PM on demo-cluster

Cmd 9

1 spark.createDataFrame(data_list, schema_2).printSchema()

root
|-- a: integer (nullable = true)
|-- b: double (nullable = true)
|-- c: string (nullable = true)
|-- d: date (nullable = true)
|-- e: timestamp (nullable = true)

Command took 0.15 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:21:32 PM on demo-cluster
```

The *createDataFrame* is even more flexible.

We used Python list and schema definition for creating a Spark Dataframe.

However, the create Dataframe can also take the following.

1. List of DataFrame Row
2. Pandas DataFrame
3. Spark RDD

List of DataFrame Row:

I am importing the Row class from the pyspark.sql package. You can create a list of rows and use them to create a Dataframe programmatically. Each Row is a comma-separated list of column names and column value pairs. Once you have the row list, you can use it in the *createDataFrame()* method as we used the python list.



```
06-creating-dataframes Python

demo-cluster
|-- c: string (nullable = true)
|-- d: date (nullable = true)
|-- e: timestamp (nullable = true)

Command took 0.15 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:21:32 PM on demo-cluster

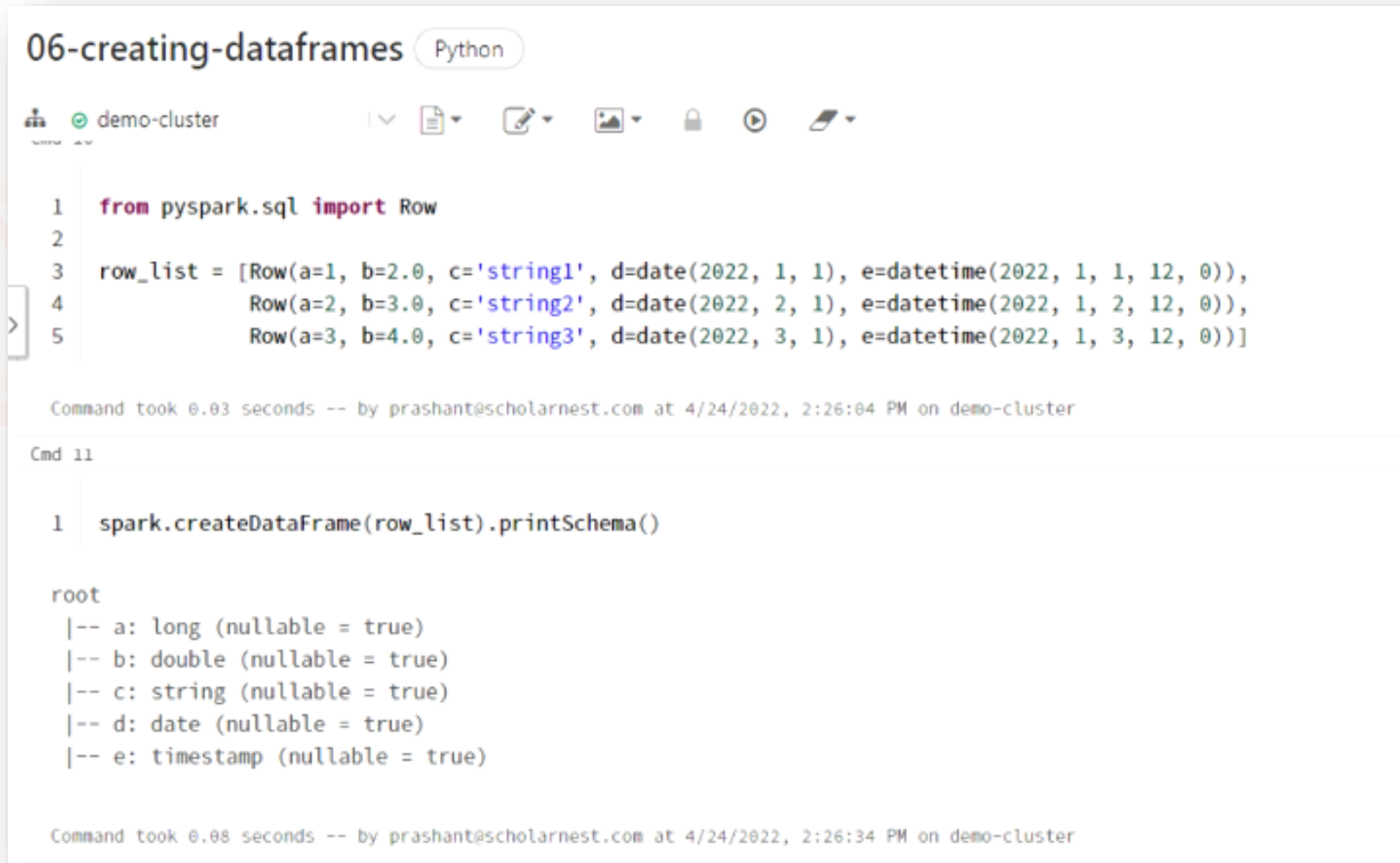
> Cmd 10

1 from pyspark.sql import Row
2
3 row_list = [Row(a=1, b=2.0, c='string1', d=date(2022, 1, 1), e=datetime(2022, 1, 1, 12, 0)),
4             Row(a=2, b=3.0, c='string2', d=date(2022, 2, 1), e=datetime(2022, 1, 2, 12, 0)),
5             Row(a=3, b=4.0, c='string3', d=date(2022, 3, 1), e=datetime(2022, 1, 3, 12, 0))]

Command took 0.03 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:26:04 PM on demo-cluster
```

List of DataFrame Row:

You can print the schema to see that row list comes with the column names, so the *createDataFrame()* method takes the correct names. However, it will infer the schema. But you can also pass the schema to the *createDataFrame()* and enforce the data types.



```
06-creating-dataframes Python

demo-cluster

1 from pyspark.sql import Row
2
3 row_list = [Row(a=1, b=2.0, c='string1', d=date(2022, 1, 1), e=datetime(2022, 1, 1, 12, 0)),
4             Row(a=2, b=3.0, c='string2', d=date(2022, 2, 1), e=datetime(2022, 1, 2, 12, 0)),
5             Row(a=3, b=4.0, c='string3', d=date(2022, 3, 1), e=datetime(2022, 1, 3, 12, 0))]

Command took 0.03 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:26:04 PM on demo-cluster

Cmd 11

1 spark.createDataFrame(row_list).printSchema()

root
|-- a: long (nullable = true)
|-- b: double (nullable = true)
|-- c: string (nullable = true)
|-- d: date (nullable = true)
|-- e: timestamp (nullable = true)

Command took 0.08 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:26:34 PM on demo-cluster
```

Pandas DataFrame:

So I am importing the panda's package and defining a panda Dataframe. You might not have worked with pandas package, and that's fine. We are just showing an example, so you know how to convert a pandas Dataframe into a Spark Dataframe.



The screenshot shows a Jupyter Notebook interface with a title bar "06-creating-dataframes" and a "Python" language selector. The notebook contains a code cell with the following Python code:

```
-- c: string (nullable = true)
-- d: date (nullable = true)
-- e: timestamp (nullable = true)
```

Below the code cell, a status bar indicates: "Command took 0.08 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:26:34 PM on demo-cluster".

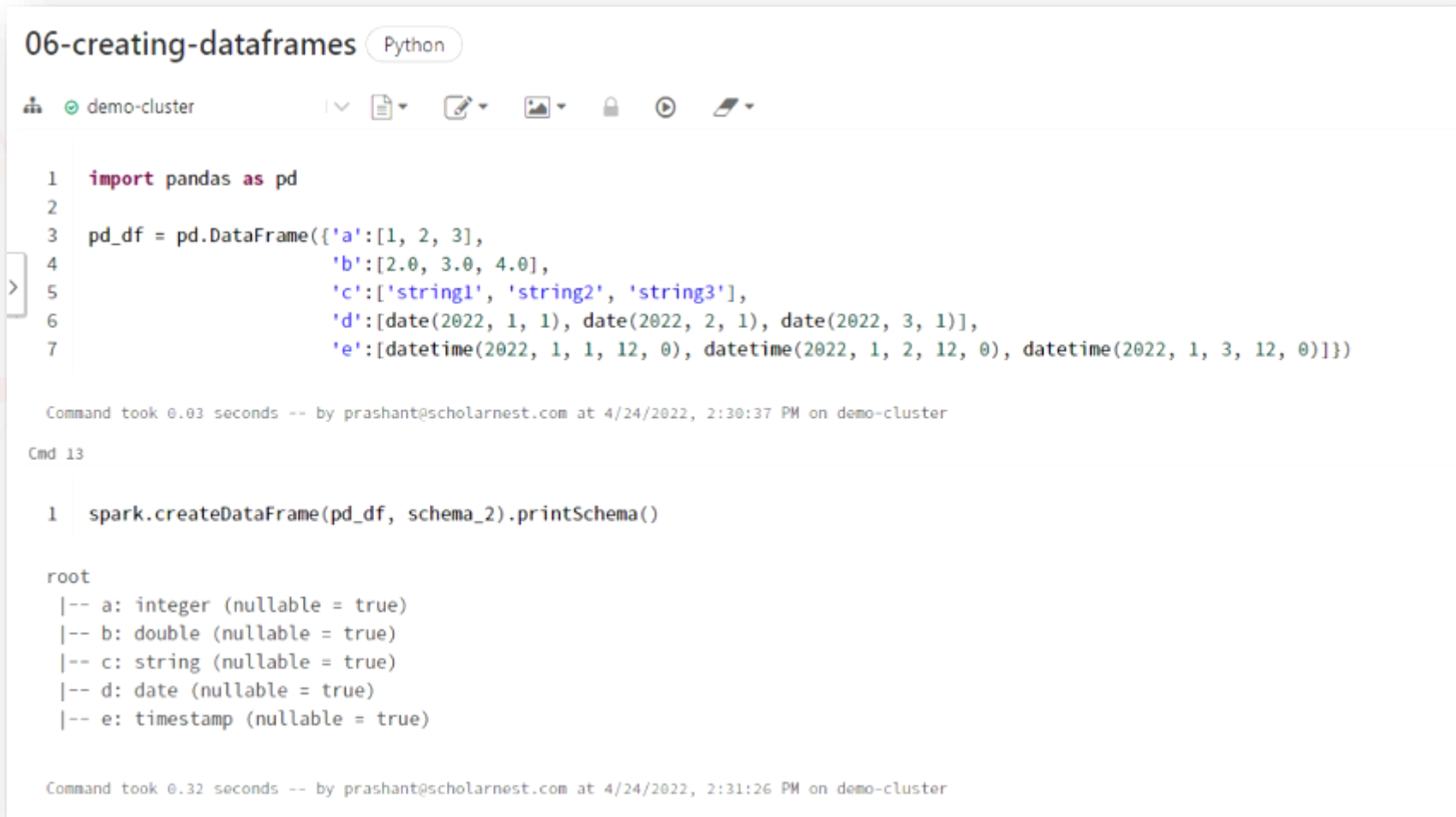
The next cell is labeled "Cmd 12" and contains the following Python code:

```
1 import pandas as pd
2
3 pd_df = pd.DataFrame({'a':[1, 2, 3],
4                       'b':[2.0, 3.0, 4.0],
5                       'c':['string1', 'string2', 'string3'],
6                       'd':[date(2022, 1, 1), date(2022, 2, 1), date(2022, 3, 1)],
7                       'e':[datetime(2022, 1, 1, 12, 0), datetime(2022, 1, 2, 12, 0), datetime(2022, 1, 3, 12, 0)]})
```

Below this code cell, a status bar indicates: "Command took 0.03 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:30:37 PM on demo-cluster".

Pandas DataFrame:

You can use the `createDataFrame()` to print the schema. I used `schema_2` so I could enforce the schema on this Dataframe. However, the schema is an optional argument. If you do not supply the schema, the spark will infer it from the given values.



```
06-creating-dataframes Python

demo-cluster

1 import pandas as pd
2
3 pd_df = pd.DataFrame({'a':[1, 2, 3],
4                       'b':[2.0, 3.0, 4.0],
5                       'c':['string1', 'string2', 'string3'],
6                       'd':[date(2022, 1, 1), date(2022, 2, 1), date(2022, 3, 1)],
7                       'e':[datetime(2022, 1, 1, 12, 0), datetime(2022, 1, 2, 12, 0), datetime(2022, 1, 3, 12, 0)]})

Command took 0.03 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:30:37 PM on demo-cluster

Cmd 13

1 spark.createDataFrame(pd_df, schema_2).printSchema()

root
 |-- a: integer (nullable = true)
 |-- b: double (nullable = true)
 |-- c: string (nullable = true)
 |-- d: date (nullable = true)
 |-- e: timestamp (nullable = true)

Command took 0.32 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:31:26 PM on demo-cluster
```

Spark RDD:

Spark core package defines an RDD data structure. The full form of the RDD is Resilient Distributed Dataset. However, it is as simple as an in-memory distributed data structure. In the earlier versions of Spark, we used RDDs for data processing. However, the later versions of Spark came up with the Dataframe. Dataframe is also a distributed data structure. It is built on top of the RDD. However, Dataframe is optimized and more efficient than the RDD.

The RDD cannot have a schema. It is schema-less. But Dataframe must have a schema. You cannot create a schema-less Dataframe. It must have a schema. If you do not provide a schema, Spark will apply some default schema. We saw the same earlier, *createDataFrame()* method infers the data types and adds some default column names to create a default schema.

Hence, Spark Dataframe is a more advanced, optimized, and schema-full data structure. We recommend using Dataframes as much as possible. Spark RDD is part of the Spark core package, and it is schema-less. We should avoid using RDD and use Dataframes.

Spark RDD:

Here is the code for manually creating an RDD. I am using the spark session to get the SparkContext. Why SparkContext? Because SparkContext is my gateway to the Spark core package. Then we use the parallelize() method to create an RDD.

Once you have an RDD, you can impose a schema and convert it to a Dataframe.



The screenshot shows a Databricks notebook interface. At the top, the notebook title is "06-creating-dataframes" and the language is set to "Python". Below the title, there's a toolbar with icons for cluster management, file operations, and execution. The main code area shows a Python script that defines a schema and creates an RDD. The schema is defined with three fields: a string (nullable), a date (nullable), and a timestamp (nullable). The RDD is created by parallelizing a list of three tuples, each containing an integer, a string, a date, and a timestamp. The execution results show that the command took 0.32 seconds to run. Below the code, there's a command prompt showing the execution of the code, which took 0.05 seconds to run.

```
06-creating-dataframes Python

demo-cluster
|-- c: string (nullable = true)
|-- d: date (nullable = true)
|-- e: timestamp (nullable = true)

Command took 0.32 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:31:26 PM on demo-cluster

Cmd 14

1 rdd = spark.sparkContext.parallelize([(1, 2., 'string1', date(2022, 1, 1), datetime(2022, 1, 1, 12, 0)),
2                                     (2, 3., 'string2', date(2022, 2, 1), datetime(2022, 1, 2, 12, 0)),
3                                     (3, 4., 'string3', date(2022, 3, 1), datetime(2022, 1, 3, 12, 0))])

Command took 0.05 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:34:57 PM on demo-cluster
```

Spark RDD:

You can see the desired output if you print the schema.



The screenshot shows a Databricks notebook interface. At the top, the notebook is titled "06-creating-dataframes" and the language is set to "Python". Below the title bar, there's a toolbar with icons for various actions. The main area displays two code cells. The first cell, labeled "Cmd 14", contains a Spark RDD creation command using `spark.sparkContext.parallelize` with three rows of data. The second cell, labeled "Cmd 15", contains a command to create a DataFrame from the RDD and print its schema. The output of the second command shows the schema details for each column: `a` is a long, `b` is a double, `c` is a string, `d` is a date, and `e` is a timestamp, all nullable.

```
06-creating-dataframes Python

demo-cluster

Cmd 14

1 rdd = spark.sparkContext.parallelize([(1, 2., 'string1', date(2022, 1, 1), datetime(2022, 1, 1, 12, 0)),
2                                     (2, 3., 'string2', date(2022, 2, 1), datetime(2022, 1, 2, 12, 0)),
3                                     (3, 4., 'string3', date(2022, 3, 1), datetime(2022, 1, 3, 12, 0))])

Command took 0.05 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:34:57 PM on demo-cluster

Cmd 15

1 spark.createDataFrame(rdd, schema_1).printSchema()

(2) Spark Jobs

root
|-- a: long (nullable = true)
|-- b: double (nullable = true)
|-- c: string (nullable = true)
|-- d: date (nullable = true)
|-- e: timestamp (nullable = true)

Command took 0.70 seconds -- by prashant@scholarnest.com at 4/24/2022, 2:36:21 PM on demo-cluster
```



Thank You
ScholarNest Technologies Pvt Ltd.
www.scholarnest.com